

Comunicações Por Computador - Trabalho Prático 2

Cátia Eira (a107382), Filipa Cosquete (a106837), Fábio Silva (a82331)

Grupo 2

Índice

1. Introdução.....	3
2. Diagrama de Classes.....	4
3. Formato das Mensagens e Protocolos.....	5
3.1. <i>Package</i> (Pacote de Envio).....	5
3.2. <i>Message</i> (Mensagem Base).....	5
3.3. <i>MessageUDP</i> (Mensagem com Metadados de Fiabilidade).....	5
3.4. Tipos de Dados de Mensagem (<i>MessageData</i>).....	6
3.4.1. <i>MissionMessage</i>	6
3.4.2. <i>RequestMission</i>	6
3.4.3. <i>UpdateMission</i>	6
3.4.4. <i>RoverInitMessage</i>	7
3.4.5. <i>RoverTelemetryMessage</i>	7
3.4.6. <i>ACKMessage</i>	7
4. Mission Link.....	8
4.1. Gestão de ACKs.....	8
4.2. Gestão de fragmentos.....	9
4.3. Retransmissões.....	9
4.4. Duplicados.....	9
5. Telemetry Stream.....	11
6. Simulação do Rover e missões.....	12
6.1. Inicialização do Rover.....	12
6.2. Criação das Missões.....	12
6.3. Simulação do estado do Rover.....	12
6.4. Simulação das Missões.....	13
7. API de Observação da Nave-Mãe.....	14
7.1. <i>Endpoints</i> da API.....	14
7.2. Estrutura das Mensagens.....	14
8. Nó Ground Control.....	15
9. Testes e exibição do funcionamento.....	18
10. Conclusão.....	21

1. Introdução

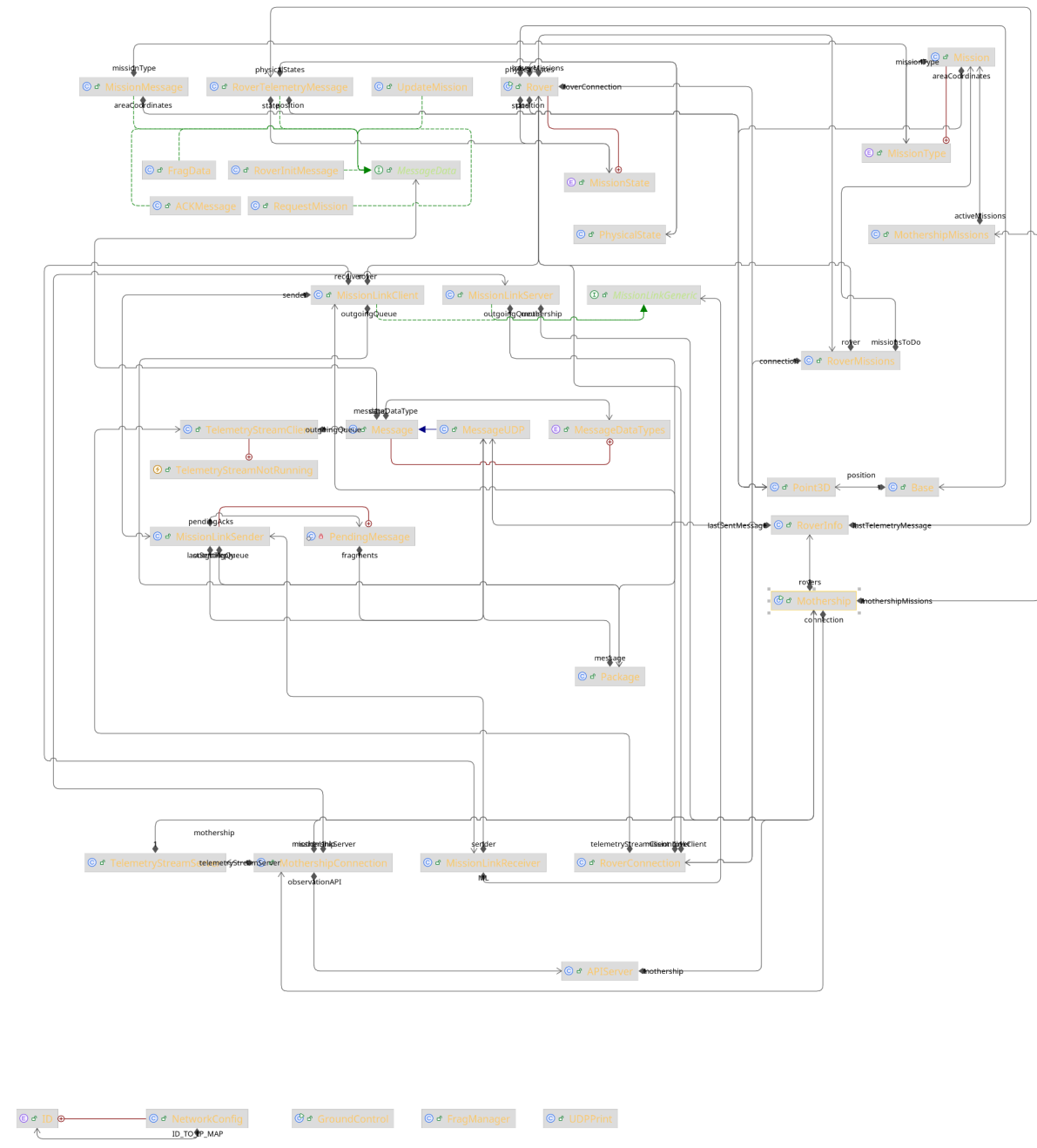
Este trabalho prático simula um sistema de comunicações entre uma Nave-Mãe e múltiplos Rovers autónomos. São utilizados dois canais de comunicação: o *Mission Link*, baseado em UDP, e o *Telemetry Stream*, baseado em TCP.

O *Mission Link* conta ainda com a implementação de uma camada de fiabilidade, incluindo mecanismos de controlo de ACKs, fragmentação, retransmissões, e detecção de mensagens duplicadas, o que permite lidar com perdas de pacotes.

Cabe à Nave-Mãe coordenar todo o processo de criação e atribuição de missões aos Rovers, disponibilizando o estado do sistema através de uma API, consumida pelo *Ground Control*.

É incluído na pasta um ficheiro *README.md* que detalha como compilar e correr as diferentes partes do programa.

2. Diagrama de Classes



3. Formato das Mensagens e Protocolos

3.1. *Package* (Pacote de Envio)

O *Package* é a unidade de transporte mais externa, utilizada para endereçar a mensagem na rede, usada na *MissionLink*.

Campos

- **IP do Recetor:** O endereço IP de destino (*Mothership* ou *Rover*).
- **Porta do Recetor:** A porta de rede de destino.
- **Mensagem UDP:** O conteúdo da mensagem (a estrutura *MessageUDP*) a ser transmitida.

3.2. *Message* (Mensagem Base)

A *Message* é a estrutura fundamental que contém os dados e metadados essenciais para a aplicação. É a estrutura enviada na conexão TCP.

Campos

- **ID da Mensagem:** Um identificador único para esta instância de mensagem.
- **Tipo dos Dados da Mensagem:** Um campo de enumeração que especifica qual o tipo da mensagem que está contido no campo Dados da Mensagem (ex: *MISSION*, *ACK*, *ROVER_TELEMETRY*).
- **Dados da Mensagem:** Os dados serializados (*MissionMessage*, *RequestMission*, etc.) que transportam a informação útil.

3.3. *MessageUDP* (Mensagem com Metadados de Fiabilidade)

O *MessageUDP* expande a *Message* base e a adiciona os metadados necessários para implementar uma camada de fiabilidade sobre o protocolo UDP, de modo a tratar a perda e reordenação de pacotes.

Campos

- **Todos os campos de uma Message**
- **Número de Sequência:** Utilizado para ordenar pacotes e identificar duplicados.
- **Número ACK:** Utilizado para acusar o recebimento de uma mensagem anterior do recetor.
- **ID do Fragmento:** Identificador para reconstruir a mensagem original.
- **Index do Fragmento:** Posição do fragmento dentro da mensagem original.
- **Número Total de Fragmentos:** O número total de partes que compõem a mensagem original.

3.4. Tipos de Dados de Mensagem (MessageData)

O campo Dados da Mensagem pode conter os seguintes tipos de dados:

- *MISSION*
- *REQUEST_MISSION*
- *MISSION_UPDATE*
- *ROVER_INIT*
- *ROVER_TELEMETRY*
- *ACK*

3.4.1. *MissionMessage*

Campos

- ID da Missão
- ID do Rover
- Tipo da Missão
- Coordenadas da Área de Missão
- Raio da Área de Missão
- Tempo de Missão (Duração total)
- Tempo de Atualização (Frequência de envio de updates)
- Flag de Urgência
- Flag de Conclusão

3.4.2. *RequestMission*

Utilizada pelo Rover para solicitar uma nova missão à Nave-Mãe.

Campos

- ID do Rover

3.4.3. *UpdateMission*

Utilizada pelo Rover para enviar o progresso atual de uma missão em andamento para a Nave-Mãe.

Campos

- ID da Missão
- ID do Rover
- Nível de Conclusão (0 a 100)
- Dados Extra (Geralmente vazio)

3.4.4. *RoverInitMessage*

Utilizada pelo Rover para iniciar o processo de conexão e obter seu ID único da Nave-Mãe.

Campos

- ID do Rover

3.4.5. *RoverTelemetryMessage*

Utilizada pelo Rover para enviar o seu estado completo e atual à Nave-Mãe, usado para monitoramento.

Campos

- ID do Rover
- Posição Atual (Coordenadas X, Y, Z)
- Estado Operacional (ex.: *IDLE*, *CHARGING*, *ON_THE_WAY*)
- Condição dos Componentes Físicos (Lista de estados de peças)
- Nível da Bateria
- Inventário (Lista de itens colectados)

3.4.6. *ACKMessage*

Utilizada para confirmar o recebimento de uma mensagem, importante em protocolos que exigem confirmação de entrega (como o UDP, no *MissionLink*).

Campos

- Número de Sequência da mensagem confirmada

4. Mission Link

O Mission Link foi dividido em duas partes principais: *MissionLinkReceiver* (*MLReceiver*), e *MissionLinkSender* (*MLSender*), responsáveis por aplicar os protocolos de receber e enviar mensagens, respetivamente. Estes componentes trabalham em conjunto tanto para aplicar a lógica de controlo de ACKs, de forma a assegurar que uma mensagem enviada e perdida é retransmitida, como para cuidar da fragmentação, de forma a prevenir que o recetor receba uma mensagem maior do que o seu *buffer* permite, dividindo o *payload* da mensagem em vários fragmentos.

Ambas as classes são aplicadas no *MissionLinkClient* e *MissionLinkServer*, chamadas pelos Rovers e pela Nave-Mãe, respetivamente, de forma a criarem e gerirem os respetivos protocolos, onde a única diferença é a forma de processar as mensagens recebidas e as respostas que estas precisam (a Nave-Mãe regista um *ROVER_INIT* de forma diferente de um Rover, por exemplo).

4.1. Gestão de ACKs

O *MLSender*, classe responsável pelo envio das mensagens, mantém um mapa de controlo com as mensagens que aguardam confirmação (ACK), de forma a garantir o funcionamento correto mesmo quando comunica com vários recetores em simultâneo. Para cada mensagem enviada que requer confirmação, é registado o seu estado, incluindo o número de tentativas realizadas, e o instante do último envio.

O envio das mensagens é feito de forma contínua: o *MLSender* retira mensagens da fila de saída, fragmenta-as se necessário, e envia-as para a rede. No caso de mensagens que não são ACKs puros, estas são adicionadas ao mapa de mensagens pendentes logo após o envio. As mensagens que correspondem apenas a ACKs são enviadas diretamente, não sendo registadas nem aguardando o retorno de uma confirmação.

A gestão dos *timeouts* e das retransmissões é realizada por um serviço separado, que periodicamente percorre o mapa de mensagens pendentes. Para cada entrada, verifica se o tempo máximo de espera por um ACK foi ultrapassado. Caso isso aconteça e o número máximo de tentativas ainda não tenha sido atingido, a mensagem é retransmitida e o seu estado atualizado. Se o número máximo de retransmissões for excedido, o envio da mensagem é abandonado, de modo a evitar o desperdício de recursos do sistema.

O *MLReceiver*, ao receber uma mensagem, analisa o número de ACK associado. Se o valor recebido for válido e corresponder a uma mensagem previamente enviada para o endereço de origem, essa mensagem é considerada confirmada e removida do mapa de mensagens pendentes no *MLSender*. O ACK esperado corresponde à soma do número de sequência da mensagem com o tamanho do respetivo *payload*.

Desta forma, o sistema garante a fiabilidade na entrega das mensagens sem bloquear o processo de envio, assegurando uma comunicação eficiente e robusta entre múltiplos clientes.

4.2. Gestão de fragmentos

Para que pudéssemos gerir corretamente a fragmentação, adicionámos vários campos à nossa mensagem UDP, tais como o *fragmentID*, o *fragmentIndex* e o *totalFragments*, cada um desempenhando o seu papel na lógica de fragmentação implementada. Foi também criada uma classe *fragManager* que possui a lógica fragmentação e remontagem das mensagens.

Quando o programa se prepara para enviar uma mensagem, precisa de saber se esta não excede o tamanho máximo permitido. A mensagem é convertida em bytes, e, caso o limite seja excedido, é calculado o *fragmentID* para essa mensagem, o qual vai servir para identificar todos os fragmentos pertencentes à mesma. A partir daí, o *payload* é dividido em partes iguais, às quais é adicionado o header original. Estas novas mensagens são enviadas sequencialmente, cabendo ao recetor montar a mensagem original.

O recetor tem um *Hashmap* onde guarda as mensagens recebidas com o ID correspondente ao grupo de fragmentos. Quando recebe uma mensagem, verifica se é um fragmento. Em caso afirmativo, verifica se este já existe no seu *Hashmap*, para que não guarde fragmentos duplicados. Depois, verifica se já possui todos os fragmentos da mensagem, e remonta a mesma.

O processo de remontagem acontece da seguinte forma: o método pega na lista de fragmentos, e ordena-a de acordo com os índices dos fragmentos. Depois, cria um array de bytes, onde escreve o payload de cada fragmento. Quando todos os fragmentos tiverem sido escritos, o array é convertido numa mensagem UDP, que é retornada ao recetor.

4.3. Retransmissões

O mecanismo que implementámos baseia-se em *stop-and-wait*. Para tal, o *sender* possui um *Hashmap* onde guarda o número do ACK que cada mensagem enviada espera receber. Isto permite que cada rover tenha o seu “espaço dedicado” dentro do *Hashmap*, ou seja, que vários rovers consigam enviar mensagens ao mesmo tempo para a nave mãe, sem que ela as confunda.

Quando é recebido um ACK, é verificado se este existe no *Hashmap* e, no caso de existir, a *thread* é desbloqueada.

Se o ACK não chegar, ou não existir no *Hashmap*, a *thread* atinge um *timeout* e retransmite a mensagem. Existe também um limite após o qual não há mais retransmissões da mesma mensagem e a mesma é dada como perdida.

4.4. Duplicados

Na implementação feita temos dois cenários: quando uma mensagem é fragmentada, e quando não o é.

No primeiro caso, tal como explicado anteriormente, cada fragmento é verificado individualmente antes de ser inserido no *HashMap* do recetor. Esta verificação garante que apenas fragmentos válidos e ainda não recebidos são considerados.

No segundo caso, é comparado o número de sequência da mensagem recebida com o último número de sequência que foi recebido. Se for maior, é porque foi recebida uma mensagem nova, e se for menor ou igual, é porque é um duplicado.

O comportamento do recetor perante duplicados é enviar um ACK, para garantir que quem enviou a mensagem não a retransmita mais. Isto acontece quando há perdas de pacotes na rede e a mensagem é recebida, mas o seu ACK não.

Outro comportamento do recetor é enviar uma mensagem, no caso de ele receber um pedido de *INIT*, por exemplo.

Estes comportamentos diferem dependendo do recetor, Nave-Mãe ou Rover.

É de notar que quando um Rover recebe uma missão duplicada, ele ignora o conteúdo da missão, pois sabe que é um duplicado e que a mesma já foi aceite.

5. *Telemetry Stream*

Durante a inicialização, a Nave-Mãe abre um *ServerSocket* numa porta pré-definida e conhecida, que permite aos Rovers conectarem-se. Quando um Rover está no processo de iniciar a conexão com a Nave-Mãe, numa *thread* à parte, estabelece um *socket* TCP, que irá esperar por mensagens a serem enviadas. A cada 30 segundos, o Rover cria uma mensagem de telemetria, que contém os dados relevantes sobre o seu estado, e adiciona-la à *queue* de mensagens a serem enviadas pelo *TelemetryStream*. Como TCP é um protocolo seguro, não foi necessário criar medidas de gestão de erros, controlo, ou congestão.

A Nave-Mãe lê e guarda a mensagem recebida, mantendo-a associada ao ID do Rover que a enviou, de modo a que o *Ground Control* tenha acesso à última mensagem de telemetria enviada por cada Rover. Caso o Rover se desconecte do *socket*, pela natureza segura do TCP, a Nave-Mãe saberá de imediato, e removerá o Rover da lista de Rovers ativos.

6. Simulação do Rover e missões

6.1. Inicialização do Rover

O processo de inicialização de um Rover é uma troca de mensagens em três etapas com a Nave-Mãe, semelhante a um *3-Way-Handshake*, que visa garantir que o Rover obtenha um identificador único para operação.

Primeiramente, o Rover envia uma mensagem de *ROVER_INIT* que contém o ID que pretende utilizar. A Nave-Mãe recebe esta solicitação e procede à validação do ID, verificando se não existe nenhum outro Rover ativo já registado com a mesma identificação.

Em seguida, a Nave-Mãe responde ao Rover, ou confirmando o ID proposto por ser válido e único, ou informando que é inválido (ao indicar o valor do ID como -1) se já existir um Rover com o mesmo ID.

Finalmente, o Rover envia uma mensagem de ACK para a Nave-Mãe, a confirmar que recebeu a resposta. A partir desta confirmação, se a resposta da Nave-Mãe tiver sido de validação, o Rover prossegue com o seu funcionamento normal e passa a operar com o ID atribuído. Caso contrário, encerra de imediato as suas funções.

6.2. Criação das Missões

Após a inicialização, a Nave-Mãe começa a criar missões com parâmetros aleatórios e sem Rovers definidos, para simular missões que qualquer Rover poderia fazer. Estas são guardadas numa *queue* de prioridade de missões à espera para serem atribuídas e enviadas (ordenadas por missões urgentes e não urgentes). Quando existem Rovers ativos (ou seja, inicializados e conectados pela telemetria TCP), começa a criar missões para Rovers específicos, que são enviadas de imediato.

Após um Rover se conectar à Nave-Mãe, este envia uma mensagem a pedir uma missão, para informar a Nave-Mãe de que está disponível e pronto para aceitar trabalho. Caso a fila de espera de missões esteja vazia, a Nave-Mãe cria uma missão nova. Caso haja missões, escolhe uma, atribuindo-a ao Rover, declarando-a como uma missão ativa, e, finalmente, enviando a missão ao Rover através do *MissionLink*.

6.3. Simulação do estado do Rover

O Rover começa em modo *IDLE* (em descanso), na sua base. Sempre que decorre alguma ação (carregamento da bateria, movimentação, início de missão, etc.), é calculado o tempo estimado de duração dessa ação, e o Rover atualiza o seu estado de acordo, a cada segundo, até o tempo terminar. Esta forma de atualizar o estado do Rover foi escolhida para permitir que as mensagens de telemetria e atualizações de missão sejam o mais fiéis possíveis, e para garantir que o Rover não fica preso num só estado.

Sempre que o Rover está na base, passa por várias verificações antes de pedir ou iniciar uma missão. Verifica se precisa de arranjo (caso alguma das suas partes esteja com condição igual ou inferior a 10%) e, em caso afirmativo, passa a estar em modo *ERROR*, onde irá começar a arranjar todas as suas partes. Se tiver itens no seu inventário irá deixá-las na base, e, caso não tenha nenhuma missão na sua lista e tiver bateria inferior a 50%, passa a estar em modo *CHARGING* (carregamento), de modo a estar com a bateria cheia quando pedir uma missão.

6.4. Simulação das Missões

Após todas as verificações, o Rover está pronto para fazer uma missão. Escolhe então uma missão da sua lista, dando prioridade às missões urgentes, e, caso a lista esteja vazia, pede uma à Nave-Mãe, ficando parado até receber uma resposta.

Quando houver uma missão escolhida, o Rover calcula a quantidade de bateria que será gasta para a execução da missão. Este cálculo é baseado na distância a que o Rover se encontra do local da missão, no seu tempo de duração e no tipo da missão. Se a carga atual for inferior à estimada, o Rover recarrega primeiro. No entanto, se a carga necessária for superior a 100%, o Rover nunca conseguirá concluir a missão, e anuncia à Nave-Mãe que a irá descartar. A mensagem usada é um *MissionUpdate*, com a percentagem de completude a -1. Ao receber esta mensagem, a Nave-Mãe move a missão da lista de missões ativas para as missões descartadas, enquanto o Rover reinicia o processo de escolher uma missão.

Finalmente, quando o Rover encontra uma missão que consegue realizar, designa-a como a sua missão atual e o seu estado muda para *ON_THE_WAY* (a caminho). Neste estado, a posição do Rover altera-se a cada segundo, de acordo com o tempo passado e com a posição final. Quando chega finalmente ao destino, passa a estar no estado *IN_MISSION*. No caso de não haver uma missão atual definida, assume-se que chegou à base, e passa a *IDLE*.

Quando em missão, envia mensagens de atualização de missão enquanto o tempo estimado não se esgotar, e, caso a missão envolva itens, como a coleção de amostras, existe uma chance de obter novos itens a cada segundo. Para enviar atualizações à Nave-Mãe é criado um *Scheduler*, que envia uma nova mensagem de acordo com o tempo definido pela Nave-Mãe na missão. A mensagem detalha a percentagem de completude, calculada com base no tempo atual e no momento em que a missão começou. No caso da missão ser de exploração, são enviados dados extra em cada mensagem, simbólicos de um vídeo ou de uma imagem que o Rover terá tirado ao explorar, com o intuito de mostrar o controlo de fragmentação do UDP.

Quando o Rover termina a missão, envia uma última mensagem com a completude a 100%, e decide se escolhe uma nova missão ou se volta para a base, de acordo com a bateria que lhe resta, o espaço livre do inventário, e se tem mais missões na sua lista. A Nave-Mãe, ao receber a última mensagem, marca a missão como completa e move-a para a lista das missões completas.

Será de notar que a bateria decresce quando está em missão ou a caminho, com diferentes níveis de consumo de acordo com o tipo da missão (coleccionar amostras da atmosfera gasta pouca energia, explorar gasta bastante). A condição física das partes diminui de forma aleatória a cada missão feita.

7. API de Observação da Nave-Mãe

A API disponibilizada pela Nave-Mãe é implementada através de um servidor HTTP simples, recorrendo à biblioteca `com.sun.net.httpserver.HttpServer`, e permite monitorizar o estado dos Rovers ativos, missões e mensagens de telemetria.

Esta API é exclusivamente de leitura, e tem como objetivo fornecer uma visão do estado do sistema para entidades externas como o *Ground Control*. É iniciada juntamente com o *MissionLink* e o *TelemetryStream*, aquando do "arranque" da Nave-Mãe.

7.1. Endpoints da API

A API define os seguintes *endpoints* HTTP:

Endpoint	Descrição
/rovers	Retorna informações sobre todos os Rovers atualmente conectados à Nave-Mãe.
/missions/active	Retorna informações sobre as missões atualmente em execução.
/missions/past	Retorna informações sobre missões concluídas.
/missions/future	Retorna informações sobre missões ainda não atribuídas.
/telemetry	Retorna as últimas mensagens de telemetria enviadas pelos Rovers.

7.2. Estrutura das Mensagens

As mensagens são transmitidas em formato de texto simples. Esta decisão fez com que não fosse necessário recorrer a bibliotecas externas para conversão de objetos, o que simplificou a arquitetura e reduziu a dependência de componentes adicionais.

Os objetos relevantes (*RoverInfo*, *Mission*, e *RoverTelemetryMessage*) implementam o método `toStringForAPI()`, que converte os mesmos numa representação textual já estruturada para exposição através da API.

8. Nó Ground Control

O programa para o nó Ground Control é uma aplicação de consola que atua como cliente do servidor da API exposta pela Nave-Mãe, permitindo a um operador humano consultar, de forma interativa, o estado do sistema. É iniciado correndo o script `groundcontrol.sh` no respetivo nó da infraestrutura.

Após o arranque, a aplicação estabelece ligação ao servidor HTTP da Nave-Mãe e apresenta um menu interativo, navegável através de opções numéricas. Este menu permite ao utilizador:

1. Consultar a lista de Rovers ativos

```
ROVER INFO
+-----+
| Rover 1: |
+-----+
| IP address -> /10.0.3.10 |
| Status -> ON_THE_WAY |
| Battery -> 85.06% |
| Last Active -> 14 seconds ago |
+-----+
| Rover 2: |
+-----+
| IP address -> /10.0.4.10 |
| Status -> IN_MISSION |
| Battery -> 57.19% |
| Last Active -> 17 seconds ago |
+-----+
```

2. Consultar a lista de missões ativas

```
ACTIVE MISSIONS
+-----+
| Mission 1: |
+-----+
| Rover -> 1 |
| Mission Type -> COLLECT_ROCKS |
| Coordinates -> (0.0, 4.0, 3.0) |
| Radius -> 6 |
| Duration -> 57 |
| Update Interval -> 14 |
| Urgent -> No |
| Status -> Ongoing |
+-----+
| Mission 9: |
+-----+
| Rover -> 2 |
| Mission Type -> TEST_ATMOSPHERE |
| Coordinates -> (5.0, 19.0, 14.0) |
| Radius -> 5 |
| Duration -> 50 |
| Update Interval -> 5 |
| Urgent -> No |
| Status -> Ongoing |
+-----+
```

3. Consultar a lista de missões passadas

```
PAST MISSIONS
+-----+
| Mission 2:                                     |
+-----+
| Rover -> 2                                     |
| Mission Type -> COLLECT_ROCKS                 |
| Coordinates -> (5.0, 19.0, 11.0)              |
| Radius -> 6                                    |
| Duration -> 48                                 |
| Update Interval -> 5                         |
| Urgent -> No                                  |
| Status -> Completed                          |
+-----+
| Mission 3:                                     |
+-----+
| Rover -> 1                                     |
| Mission Type -> TEST_ATMOSPHERE               |
| Coordinates -> (17.0, 17.0, 9.0)              |
| Radius -> 10                                  |
| Duration -> 27                                 |
| Update Interval -> 2                         |
| Urgent -> No                                  |
| Status -> Completed                          |
+-----+
```

4. Consultar a lista de missões futuras

```
FUTURE MISSIONS
+-----+
| Mission 9:                                     |
+-----+
| Rover -> -1                                    |
| Mission Type -> TEST_ATMOSPHERE               |
| Coordinates -> (5.0, 19.0, 14.0)              |
| Radius -> 5                                    |
| Duration -> 50                                 |
| Update Interval -> 5                         |
| Urgent -> No                                  |
| Status -> Ongoing                            |
+-----+
```

5. Consultar as últimas mensagens de telemetria recebidas

```
MOST RECENT TELEMETRY DATA
+-----+
| Rover 1:                                     |
+-----+
| Position -> (3.0, 2.4, 3.2)                  |
| Status -> ON_THE_WAY                         |
| Battery -> 99.5%                             |
| Inventory ->                                  |
| Physical Status -> WHEELS: 94%, CAMERA: 69%  |
+-----+
| Rover 2:                                     |
+-----+
| Position -> (3.4, 9.3, 5.1)                  |
| Status -> ON_THE_WAY                         |
| Battery -> 81.5%                             |
| Inventory ->                                  |
| Physical Status -> WHEELS: 93%, CAMERA: 68%  |
+-----+
```



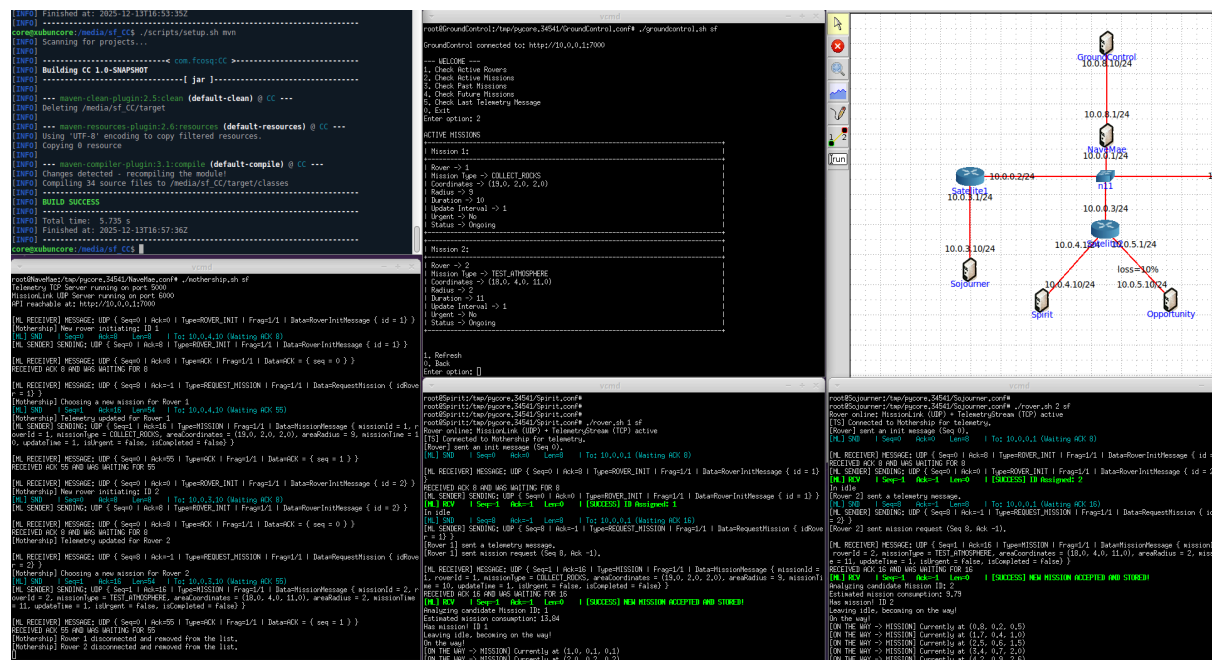
```

+-----+
| Rover 3: |
+-----+
| Position -> (0.0, 0.0, 0.0) |
| Status -> CHARGING |
| Battery -> 66.36% |
| Inventory -> |
| Physical Status -> WHEELS: 70%, CAMERA: 51% |
+-----+
| Rover 4: |
+-----+
| Position -> (5.0, 15.0, 16.0) |
| Status -> IN_MISSION |
| Battery -> 49.56% |
| Inventory -> |
| Physical Status -> WHEELS: 71%, CAMERA: 48% |
+-----+
| Rover 5: |
+-----+
| Position -> (2.1, 3.8, 2.3) |
| Status -> ON_THE_WAY |
| Battery -> 2.71% |
| Inventory -> ROCK, ROCK, ROCK, ROCK, SAMPLE |
| Physical Status -> WHEELS: 70%, CAMERA: 50% |
+-----+

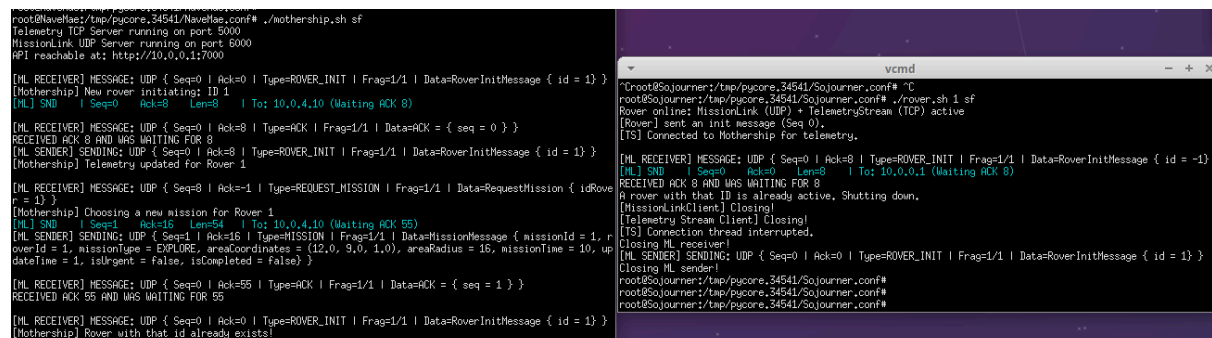
```

Cada secção possui um sub-menu próprio, que permite atualizar a informação através de uma nova requisição HTTP, ou regressar ao menu principal. A navegação é totalmente realizada através da linha de comandos.

9. Testes e exibição do funcionamento



Conexão normal sem perdas, com 2 rovers.



Tentativa de conexão com um ID repetido.

```
vcmd                                     vcmd
MissionLink UDP Server running on port 5000
Telemetry TCP Server running on port 5000
API reachable at: http://10.0.0.1:7000

[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=0 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[Mothership] New rover initiating: ID 1
[ML SND] | Seq=0 | Ack=8 | Len=8 | To: 10.0.5.10 (Waiting ACK 8)
[ML SENDER] SENDING: UDP { Seq=0 | Ack=8 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=0 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[Mothership] Rover with that id already exists!
[ML SND] | Seq=0 | Ack=8 | Len=8 | Retransmission #2 -> 10.0.5.10
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=0 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
RECEIVED ACK 0 AND WAS WAITING FOR 8
[Mothership] Rover with that id already exists!
[ML SND] | Seq=0 | Ack=8 | Len=8 | Retransmission #3 -> 10.0.5.10
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=8 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
RECEIVED ACK 8 AND WAS WAITING FOR 8
[Mothership] Telemetry updated for Rover 1
[ML SND] | Seq=1 | Ack=8 | Len=8 | Retransmission #2 -> 10.0.5.10
[ML RECEIVER] MESSAGE: UDP { Seq=1 | Ack=1 | Type=REQUEST_MISSION | Frag=1/1 | Data=RequestMission { idRover = 1 } }
RECEIVED ACK -1 AND WAS WAITING FOR 9
[Mothership] Choosing a new mission for Rover 1
[Mothership] Telemetry updated for Rover 1
[ML SND] | Seq=1 | Ack=8 | Len=8 | Retransmission #2 -> 10.0.5.10
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=9 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 1 } }
RECEIVED ACK 9 AND WAS WAITING FOR 9
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=10 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 2 } }
RECEIVED ACK 10 AND WAS WAITING FOR 10
[ML SND] | Seq=2 | Ack=8 | Len=8 | To: 10.0.5.10 (Waiting ACK 10)
[ML SENDER] SENDING: UDP { Seq=2 | Ack=8 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[ML SND] | Seq=2 | Ack=16 | Len=64 | To: 10.0.5.10 (Waiting ACK 87)
[ML SENDER] SENDING: UDP { Seq=3 | Ack=16 | Type=MISSION | Frag=1/1 | Data=MissionMessage { missionId = 1, roverId = 1, missionType = TEST_ATMOSPHERE, areaCoordinates = (10.0, 11.0, 6.0), areaRadius = 1, missionTime = 43, updateTime = 13, isUrgent = false, isCompleted = false } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=57 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 3 } }
RECEIVED ACK 57 AND WAS WAITING FOR 57
[Mothership] Rover 1 disconnected and removed from the list.
0

root@opportunity:/tmp/gucone.34541@opportunity.conf#
root@opportunity:/tmp/gucone.34541@opportunity.conf# ./rover.sh 1 sf
Rover online: MissionLink (UDP) + TelemetryStream (TCP) active
[Rover] sent an init message (Seq 0).
[TS] Connected to Mothership for telemetry.
[ML SND] | Seq=0 | Ack=0 | Len=8 | To: 10.0.0.1 (Waiting ACK 8)
[ML SENDER] SENDING: UDP { Seq=0 | Ack=0 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[ML SND] | Seq=0 | Ack=0 | Len=8 | Retransmission #2 -> 10.0.0.1
[ML SND] | Seq=0 | Ack=0 | Len=8 | Retransmission #3 -> 10.0.0.1
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=8 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
RECEIVED ACK 8 AND WAS WAITING FOR 8
[ML RCV] | Seq=1 | Ack=1 | Len=0 | [SUCCESS] ID Assigned: 1
In idle
[Rover 1] sent a telemetry message.
[ML SND] | Seq=0 | Ack=1 | Len=8 | To: 10.0.0.1 (Waiting ACK 16)
[ML SENDER] SENDING: UDP { Seq=8 | Ack=1 | Type=REQUEST_MISSION | Frag=1/1 | Data=RequestMission { idRover = 1 } }
[Rover 1] sent mission request (Seq 8, Ack -1).
[ML RECEIVER] MESSAGE: UDP { Seq=1 | Ack=8 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
RECEIVED ACK 8 AND WAS WAITING FOR 16
[ML RECEIVER] MESSAGE: UDP { Seq=2 | Ack=8 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[ML RECEIVER] MESSAGE: UDP { Seq=3 | Ack=16 | Type=MISSION | Frag=1/1 | Data=MissionMessage { missionId = 1, roverId = 1, missionType = TEST_ATMOSPHERE, areaCoordinates = (10.0, 11.0, 6.0), areaRadius = 1, missionTime = 43, updateTime = 13, isUrgent = false, isCompleted = false } }
[ML RCV] | Seq=1 | Ack=1 | Len=0 | [SUCCESS] NEW MISSION ACCEPTED AND STORED!
Analyzing candidate Mission ID: 1
Estimated mission consumption: 24.71
Has mission! ID 1
Leaving idle, becoming on the way!
On the way!
[ON THE WAY -> MISSION] Currently at (0.6, 0.7, 0.4)
[ON THE WAY -> MISSION] Currently at (1.2, 1.4, 0.7)
[ON THE WAY -> MISSION] Currently at (1.3, 2.1, 1.1)
[ON THE WAY -> MISSION] Currently at (2.5, 2.7, 1.5)
[ON THE WAY -> MISSION] Currently at (3.1, 3.4, 1.9)
[ON THE WAY -> MISSION] Currently at (5.7, 4.1, 2.2)
[ON THE WAY -> MISSION] Currently at (4.4, 4.8, 2.6)
[ON THE WAY -> MISSION] Currently at (5.0, 5.5, 3.0)
[ON THE WAY -> MISSION] Currently at (5.6, 6.2, 3.4)
root@opportunity:/tmp/gucone.34541@opportunity.conf#
```

Inicialização do Rover com 10% perdas

```
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=0 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[Mothership] New rover initiating: ID 1
[ML SND] | Seq=0 | Ack=8 | Len=8 | To: 10.0.3.10 (Waiting ACK 8)
[ML SENDER] SENDING: UDP { Seq=0 | Ack=8 | Type=ROVER_INIT | Frag=1/1 | Data=RoverInitMessage { id = 1 } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=8 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 0 } }
RECEIVED ACK 8 AND WAS WAITING FOR 8
[Mothership] Telemetry updated for Rover 1
[ML RECEIVER] MESSAGE: UDP { Seq=8 | Ack=1 | Type=REQUEST_MISSION | Frag=1/1 | Data=RequestMission { idRover = 1 } }
[Mothership] Choosing a new mission for Rover 1
[ML SND] | Seq=1 | Ack=16 | Len=64 | To: 10.0.3.10 (Waiting ACK 55)
[ML SENDER] SENDING: UDP { Seq=1 | Ack=16 | Type=MISSION | Frag=1/1 | Data=MissionMessage { missionId = 1, roverId = 1, missionType = EXPLORE, areaCoordinates = (2.0, 15.0, 8.0), areaRadius = 22, missionTime = 48, updateTime = 6, isUrgent = false, isCompleted = false } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=55 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 1 } }
RECEIVED ACK 55 AND WAS WAITING FOR 55
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/12 | Data=FragData { Size=1400, Bytes=[0, 0, 64, 16, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=2/12 | Data=FragData { Size=1400, Bytes=[-34, -98, -10, -91, -3, 56, -34, -43, -24, -3, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=3/12 | Data=FragData { Size=1400, Bytes=[-75, -94, -6, 39, -85, -102, -119, -14, -53, 2, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=4/12 | Data=FragData { Size=1400, Bytes=[-53, -4, -124, 17, -44, -74, 83, 87, 79, -72, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=5/12 | Data=FragData { Size=1400, Bytes=[-106, 44, 29, 93, 94, 29, 79, -110, 38, -75, 1, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=6/12 | Data=FragData { Size=1400, Bytes=[-33, -3, 95, -10, -25, -57, 64, 31, 11, -25, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=7/12 | Data=FragData { Size=1400, Bytes=[-36, 127, -46, 59, -72, -59, -92, 40, -77, -31, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=8/12 | Data=FragData { Size=1400, Bytes=[102, 44, 69, 105, -18, -28, -4, 110, -20, 87, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=9/12 | Data=FragData { Size=1400, Bytes=[84, -16, -53, 11, 5, -88, 22, 54, -54, -11, 47, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=10/12 | Data=FragData { Size=1400, Bytes=[44, -39, -109, -101, -63, 90, -68, 76, 7, 48, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=11/12 | Data=FragData { Size=1400, Bytes=[79, 72, 25, 120, -39, 42, -128, -61, -18, -63, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=12/12 | Data=FragData { Size=1004, Bytes=[78, 17, -20, -26, -62, 70, 85, 54, -85, 70, 87, ...] } }
[ML SENDER] SENDING: UDP { Seq=8 | Ack=1 | Type=REQUEST_MISSION | Frag=1/1 | Data=RequestMission { idRover = 1 } }
[Rover 1] sent mission request (Seq 8, Ack -1).
[ML RECEIVER] MESSAGE: UDP { Seq=1 | Ack=16 | Type=MISSION | Frag=1/1 | Data=MissionMessage { missionId = 1, roverId = 1, missionType = EXPLORE, areaCoordinates = (2.0, 15.0, 8.0), areaRadius = 22, missionTime = 48, updateTime = 6, isUrgent = false, isCompleted = false } }
RECEIVED ACK 16 AND WAS WAITING FOR 16
[ML RCV] | Seq=1 | Ack=1 | Len=0 | [SUCCESS] NEW MISSION ACCEPTED AND STORED!
Analyzing candidate Mission ID: 1
Estimated mission consumption: 99.08
Has mission! ID 1
Leaving idle, becoming on the way!
On the way!
[ON THE WAY -> MISSION] Currently at (0.1, 0.8, 0.5)
[ON THE WAY -> MISSION] Currently at (0.5, 1.7, 1.0)
[ON THE WAY -> MISSION] Currently at (0.4, 2.5, 1.5)
[ON THE WAY -> MISSION] Currently at (0.5, 3.4, 2.1)
[ON THE WAY -> MISSION] Currently at (0.6, 4.2, 2.6)
[ON THE WAY -> MISSION] Currently at (0.8, 5.1, 3.1)
[ON THE WAY -> MISSION] Currently at (0.5, 5.9, 3.6)
[ON THE WAY -> MISSION] Currently at (1.0, 6.8, 4.2)
[ON THE WAY -> MISSION] Currently at (1.2, 7.6, 4.7)
[ON THE WAY -> MISSION] Currently at (1.3, 8.4, 5.2)
[ON THE WAY -> MISSION] Currently at (1.4, 9.3, 5.7)
[ON THE WAY -> MISSION] Currently at (1.6, 10.1, 6.2)
[ON THE WAY -> MISSION] Currently at (1.7, 11.0, 6.8)
[ON THE WAY -> MISSION] Currently at (1.8, 11.8, 7.3)
[ON THE WAY -> MISSION] Currently at (1.9, 12.7, 7.8)
[ON THE WAY -> MISSION] Currently at (2.0, 13.0, 8.0)
[ON THE WAY -> MISSION] Has arrived!
In mission
[ML SND] | Seq=0 | Ack=1 | Len=15404 | To: 10.0.0.1 (12 Fragments) (Waiting ACK 15404)
[Rover 1] sent a mission update.
[ML SENDER] SENDING: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/1 | Data=UpdateMission { idMission = 1, idRover = 1, completionLevel = 0, extraData = [97, -9, 23, -60, -80, -10, -97, 95, 87, 81, 12, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=15404 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 0 } }
RECEIVED ACK 15404 AND WAS WAITING FOR 15404
[Rover 1] sent a mission update.
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=15404 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 0 } }
RECEIVED ACK 15404 AND WAS WAITING FOR 15404
[ML SND] | Seq=0 | Ack=1 | Len=15404 | To: 10.0.0.1 (12 Fragments) (Waiting ACK 15404)
[ML SENDER] SENDING: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/1 | Data=UpdateMission { idMission = 1, idRover = 1, completionLevel = 13, extraData = [-61, -89, -82, 103, 74, -19, -21, 104, 37, 4, ...] } }
[ML SND] | Seq=0 | Ack=1 | Len=15404 | To: 10.0.0.1 (12 Fragments) (Waiting ACK 15404)
[ML SENDER] SENDING: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/1 | Data=UpdateMission { idMission = 1, idRover = 1, completionLevel = 25, extraData = [-102, -6, 84, -68, 25, 84, -108, -27, -80, -11, ...] } }
[Rover 1] sent a mission update.
```

Exemplo de fragmentação (sem perdas)

```
[ML] SND | Seq=1 | Ack=16 | Len=54 | To: 10.0.0.10 (Waiting ACK 65)
[Mothership] Telemetry updated for Rover 1
[ML SENDER] SENDING: UDP { Seq=1 | Ack=16 | Type=MISSION | Frag=1/1 | Data=MissionMessage { missionId = 1, r
overId = 1, missionType = EXPLORE, areaCoordinates = (11.0, 0.0, 19.0), areaRadius = 11, missionTime = 31, u
pdateLine = 8, isUrgent = false, isCompleted = false} }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=65 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 1 } }
RECEIVED ACK 65 AND WAS WAITING FOR 65
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/12 | Data=FragData { Size=1400, B
ytes=[0, 0, 64, 16, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=2/12 | Data=FragData { Size=1400, B
ytes=[-104, 102, -33, 122, -15, -85, -20, -49, -126, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=4/12 | Data=FragData { Size=1400, B
ytes=[-62, -30, 94, -35, -74, 39, -49, 101, -44, -24, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=5/12 | Data=FragData { Size=1400, B
ytes=[94, 115, -74, 22, -35, 73, 8, 63, -101, -47, 1, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=6/12 | Data=FragData { Size=1400, B
ytes=[-43, 87, 9, -121, -72, -108, 62, 29, 9, 76, 61, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=7/12 | Data=FragData { Size=1400, B
ytes=[-94, -82, -110, -82, -91, 16, 14, 11, 113, 102, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=9/12 | Data=FragData { Size=1400, B
ytes=[111, -96, -19, -65, 94, -109, 110, -115, 38, -... ] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=10/12 | Data=FragData { Size=1400,
Bytes=[10, -28, -56, -59, -33, -7, -38, -81, 12, 100, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=11/12 | Data=FragData { Size=1400,
Bytes=[-81, 66, -8, 106, 25, 80, -43, -42, 84, -5, 37, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=12/12 | Data=FragData { Size=1004,
Bytes=[-112, -98, -28, -77, 96, 106, -2, -90, -62, -3, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/12 | Data=FragData { Size=1400, B
ytes=[0, 0, 64, 16, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=1400 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=2/12 | Data=FragData { Size=1400, B
ytes=[-104, 102, -33, 122, -15, -85, -20, -49, -126, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=1400 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=3/12 | Data=FragData { Size=1400, B
ytes=[-35, -73, 21, -46, -6, 66, 11, -111, 7, -65, 2, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=1400 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=5/12 | Data=FragData { Size=1400, B
ytes=[94, 115, -74, 22, -35, 73, 8, 63, -101, -47, 1, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=1400 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=6/12 | Data=FragData { Size=1400, B
ytes=[-43, 87, 9, -121, -72, -108, 62, 29, 9, 76, 61, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=1400 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=7/12 | Data=FragData { Size=1400, B
ytes=[-94, -82, -110, -82, -91, 16, 14, 11, 113, 102, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=1400 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=8/12 | Data=FragData { Size=1400, B
ytes=[-33, 42, -20, -62, 58, -44, -107, 59, -49, -75, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=9/12 | Data=FragData { Size=1400, B
ytes=[111, -96, -19, -65, 94, -109, 110, -115, 38, -... ] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=11/12 | Data=FragData { Size=1400,
Bytes=[-81, 66, -8, 106, 25, 80, -43, -42, 84, -5, 37, ...] } }
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=12/12 | Data=FragData { Size=1004,
Bytes=[-112, -98, -28, -77, 96, 106, -2, -90, -62, -3, ...] } }
[Mothership] Telemetry updated for Rover 1
[ML] SND | Seq=0 | Ack=1 | Len=16404 | To: 10.0.0.1 (12 fragments) (Waiting ACK 16404)
[Rover 1] sent a mission update.
[ML SENDER] SENDING: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/1 | Data=UpdateMission { idMissio
n = 1, idRover = 1, completionLevel = 0, extradata = [-69, 58, -118, 109, 46, 122, 86, 50, -105, -40, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=16404 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=16404 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 0 } }
RECEIVED ACK 16404 AND WAS WAITING FOR 16404
[Rover 1] sent a telemetry message.
[ML] SND | Seq=0 | Ack=1 | Len=16404 | To: 10.0.0.1 (12 fragments) (Waiting ACK 16404)
[ML SENDER] SENDING: UDP { Seq=0 | Ack=1 | Type=MISSION_UPDATE | Frag=1/1 | Data=UpdateMission { idMissio
n = 1, idRover = 1, completionLevel = 26, extradata = [-79, -117, -33, -101, 89, -23, 10, 38, 115, 89, ...] } }
[ML] RCV | Seq=0 | Ack=1 | Len=16404 | [DUPLICATE] Already processed, Ignored,
[ML RECEIVER] MESSAGE: UDP { Seq=0 | Ack=16404 | Type=ACK | Frag=1/1 | Data=ACK = { seq = 0 } }
[Mothership] Telemetry updated for Rover 1
```

Exemplo de fragmentação (com perdas)

10. Conclusão

O desenvolvimento deste trabalho proporcionou um aprofundamento essencial no que diz respeito a protocolos de comunicação a nível aplicacional, com o objetivo de criar um sistema de comunicação robusto e eficiente entre um servidor e múltiplos clientes.

Foi adquirido conhecimento prático sobre como as conexões TCP são estabelecidas e mantidas, e sobre como desenvolver conexões UDP seguras através da implementação de mecanismos de gestão de controlo. Esta gestão incluiu a emissão de mensagens de ACK (*Acknowledgement*) para retransmitir mensagens em caso de falha ou perda, o tratamento de mensagens duplicadas, e a modelagem da inicialização dos Rovers através de um protocolo *3-Way-Handshake* para garantir a unicidade de identificação.

Foi também gerada a simulação do estado do Rover, que reflete as suas missões e as mensagens enviadas, de acordo com os seus eventos operacionais. Finalmente, o trabalho culminou no desenvolvimento de uma API através de um servidor HTTP, permitindo a consulta remota e a monitorização eficaz do estado operacional das unidades.

Em suma, este projeto resultou na aprendizagem de como uma arquitetura de comunicação funcional e fiável para a gestão de unidades remotas deve funcionar.